THE ISLAMIC UNIVERSITY
College of Technical Engineering
Department of Computer Techniques Engineering



LECTURE 01 · WEEK 1

Protocol Hierarchies and the OSI Reference Model

How network software is organised into layers, how layers offer services to each other, and how data is wrapped and unwrapped as it crosses the seven OSI layers.

Layering and design issues	Services, interfaces, primitives	Connection-oriented vs connectionless
The seven OSI layers	PDUs and encapsulation	OSI and TCP/IP mapping

Course	Computer Networks Protocols	**Code**	CET4102 · 6 ECTS
Stage	Fourth stage · Semester 7	**Session**	Theory lecture · Week 1
Instructor	Asst. Lect. Laith H. Alzubaidi	**Academic year**	2026 – 2027

Prepared for the fourth-stage students of the Department of Computer Techniques Engineering. Primary text: J. F. Kurose & K. W. Ross, *Computer Networking: A Top-Down Approach*, 7th ed. Recommended: D. E. Comer, *Internetworking with TCP/IP*.

LEARNING OUTCOMES

By the end of this lecture you will be able to:

1. explain what a protocol is and why network software is built in layers;
2. tell a service apart from a protocol, and use the four service primitives;
3. compare connection-oriented with connectionless service, and reliable with unreliable service;
4. state the job, PDU, typical protocols and devices of each OSI layer, and map the layers onto TCP/IP;
5. trace encapsulation and decapsulation of a message and compute its header overhead.

When you open a web page, the data must be cut into pieces, addressed, carried over cables or radio, checked for errors and put back together in order. No single program does all of this. The work is split into **layers**, and each layer follows its own set of rules, called a **protocol**.

This lecture gives the map of those layers, the OSI reference model, and the words used in every later lecture: protocol, service, peer, PDU and encapsulation. Later lectures study the layers one by one.

1.1 Protocols and layers

Think about asking a stranger for the time. You say "Excuse me", wait for "Yes?" and only then ask the time. If the reply is rude, you walk away. Both people know which messages to send, in which order, and what to do with each reply. Computers need the same agreement, written down exactly.

KEY DEFINITION: PROTOCOL

A **protocol** fixes the **format** of the messages that two or more entities exchange, the **order** in which they are exchanged, and the **actions** taken when a message is sent or received, or when another event happens (for example, a timer runs out).

Every protocol has three elements. **Syntax** is the structure of a message: which fields it has and in what order. **Semantics** is the meaning of each field and the action it causes. **Timing** says when data may be sent and how fast, so that a fast sender does not overrun a slow receiver.

The job of a network is too big for one protocol, so it is split into a stack of layers, each solving one part. Layer n uses the service of layer $n-1$ and offers a service to layer $n+1$. It hides *how* it works; the layer above only needs to know *what* it gets. The post office works the same way: you post an addressed letter, and you never talk to the truck driver, who never reads your letter.

The figure below shows five layers on two hosts. Layer n on host 1 "talks" to layer n on host 2 using the **layer-n protocol**. The two talking entities are called **peers**.

In fact, no data pass directly between peers. Each layer hands its data to the layer below, the bits cross the medium, and then they climb up the other stack. So peer communication is **virtual** (dashed arrows), while the real path goes down, across and up (solid arrow). The boundary between two neighbouring layers in one host is an **interface**.

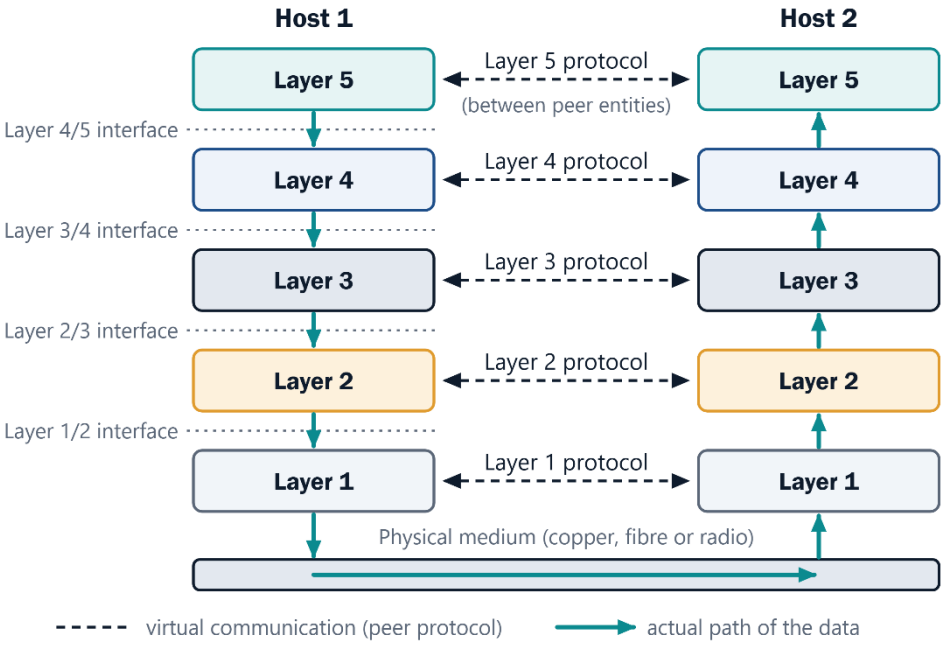


Figure 1.1 Layers, protocols and interfaces. Peers talk virtually through their protocol; the data really flow down, across the medium and up.

A set of layers and protocols is a **network architecture**. The protocols that one system uses, one per layer, form its **protocol stack**; a PC opening a web page over Ethernet uses HTTP / TCP / IP / Ethernet.

Layering has clear **benefits**: a big problem becomes several small ones; a layer can change without affecting the others (Wi-Fi can replace a cable with no change to the browser); products from different vendors work together; and faults are easier to locate. It also has **costs**: every layer adds a header (overhead) and processing time, and some jobs are repeated, such as error control in layers 2 and 4.

1.2 Design issues for the layers

Some problems appear in almost every layer. They are called **design issues**. The table lists the main ones and the layers that solve them.

Table 1.1 Design issues that appear in several layers.

Design issue	Problem it solves	Examples (layer)
Addressing	identify the sender and the receiver	MAC (L2), IP (L3), port (L4)
Connection control	open a conversation, then close it	TCP handshake (L4)
Error control	detect damaged data, then repair or resend it	CRC check (L2), retransmission (L2, L4)
Flow control	stop a fast sender flooding a slow receiver	sliding window (L2), TCP window (L4)
Message size, order	split long messages; restore their order	segmentation and sequence numbers (L4), fragmentation (L3)
Multiplexing	let many conversations share one link	TDM (L1), port numbers (L4)
Routing	choose a path when several exist	RIP, OSPF (L3)
Security	keep data secret and unchanged	IPsec (L3), TLS and SSH (above L4)

1.3 Services and service primitives

Look again at the layer figure. It shows two different relations: one between neighbouring layers in the **same** machine, and one between peers on **different** machines.

> **KEY IDEA: SERVICE VERSUS PROTOCOL**
>
> A service is **vertical**: what layer *n* does for layer *n*+1 in the same machine, through the interface. A protocol is **horizontal**: the rules followed by the peers of layer *n* on different machines. The protocol may change completely without the users noticing, as long as the service stays the same.

The layer that gives the service is the **service provider**; the layer that uses it is the **service user**. The service is reached at a service access point (SAP), which has an address, such as a port number (transport) or an IP address (network).

User and provider talk through a small set of operations called primitives. The user calls some of them to ask for something; the provider uses others to report an event to the user. In software they are usually system calls. The OSI standard ITU-T X.210 defines four types. A telephone call makes them easy to remember; the figure below shows them on a time line (time runs downwards).

Table 1.2 The four OSI service primitives.

Primitive	Direction	Meaning	Telephone call
request	user A → provider	A asks the service to act, e.g. to connect	A dials B's number
indication	provider → user B	the provider tells B about an event	B's phone rings
response	user B → provider	B answers the indication, e.g. accepts	B picks up
confirm	provider → user A	the provider tells A the result	A hears "hello"

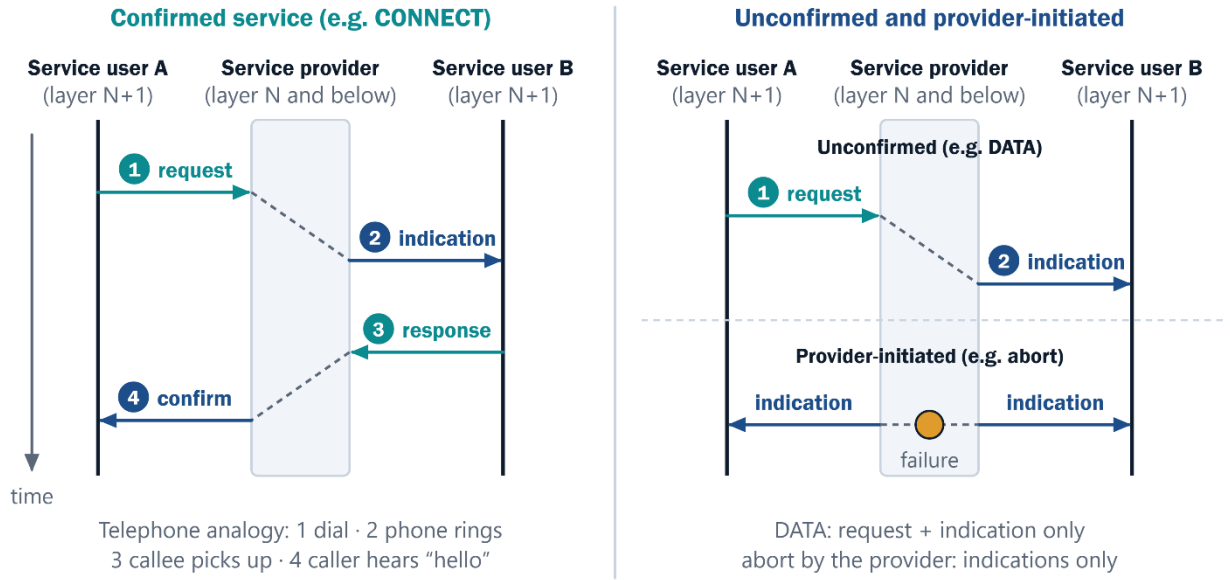


Figure 1.2 Time-sequence diagrams of a confirmed service (left) and of unconfirmed and provider-initiated services (right).

Primitives are written *layer-service.type*, e.g. `T-CONNECT.request` (T = transport). Some slides write "indicate"; the standard name is **indication**.

A **confirmed** service uses all four primitives, in the order request → indication → response → confirm, so the caller learns the result (left of the figure); connection set-up is the usual example. An **unconfirmed** service uses only request → indication, as when data are sent in a datagram service. If the provider itself aborts a connection after a failure, both users receive only indications (lower right of the figure).

Example. User A connects to user B, sends one message and hangs up. This uses the four `T-CONNECT` primitives, then `T-DATA.request` and `.indication`, then `T-DISCONNECT.request` and `.indication`: eight in total. The transport layer may also send acknowledgements, but these are protocol messages between the peers and never appear at the service interface.

1.4 Connection-oriented and connectionless service

Connection-oriented service works like a telephone call. A connection is set up, the data are sent, and the connection is released: three phases. The connection behaves like a pipe: data come out in the order they went in. Both ends keep information (state) about it. TCP is the best-known example.

Connectionless service works like the post. There is no set-up. Every message carries the full destination address and travels on its own, so messages may take different routes and arrive out of order, or not at all. Such a self-contained message is a **datagram**. IP, UDP and Ethernet are connectionless.

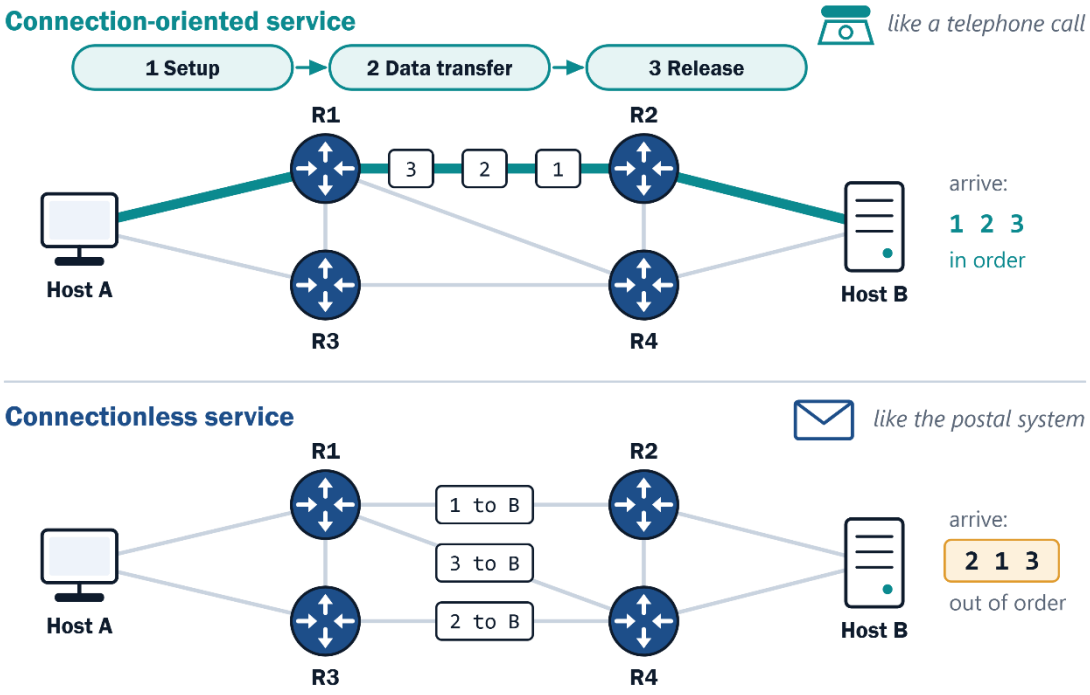


Figure 1.3 Connection-oriented service (top): here one path is fixed at set-up and packets arrive in order. Connectionless service (bottom): each packet is routed on its own and may arrive out of order.

In the top half of the figure the network itself keeps the connection and fixes the path (a *virtual-circuit* network). TCP is different: its connection lives only in the two end hosts. Its packets may take different routes through IP, and the receiving TCP puts them back in order.

Table 1.3 Connection-oriented versus connectionless service.

Aspect	Connection-oriented	Connectionless
Phases	set-up, data transfer, release	send whenever ready
Destination address	given once, at set-up	carried in every message
Order of arrival	kept	not guaranteed
Set-up delay; state	yes; state per connection	no; no state
Analogy; examples	telephone; TCP	post; IP, UDP, Ethernet

Whether a service is **reliable** is a separate question. A **reliable service** never loses data: the receiver acknowledges what arrives and the sender resends what is lost, at the cost of delay. An unreliable (*best-effort*) service makes no promise. That is often the right choice: in a live voice call, a retransmitted piece of speech would arrive too late to be played. In the Internet, TCP is connection-oriented and reliable (a file download needs it: one missing byte corrupts the file), while UDP and IP are connectionless and unreliable.

1.5 The seven layers of the OSI model

In the 1970s, each vendor used its own closed architecture, so computers from different vendors could hardly talk to each other. The International Organization for Standardization (ISO) solved this with the **Open Systems Interconnection (OSI) reference model**, published in 1984 as ISO 7498. "Open" means that any system following the standards can connect to any other.

The block of data that a layer exchanges with its peer is called a **protocol data unit (PDU)**. Each layer gives its PDU its own name, shown in the right-hand column of the figure below; the last section of this lecture looks inside a PDU.

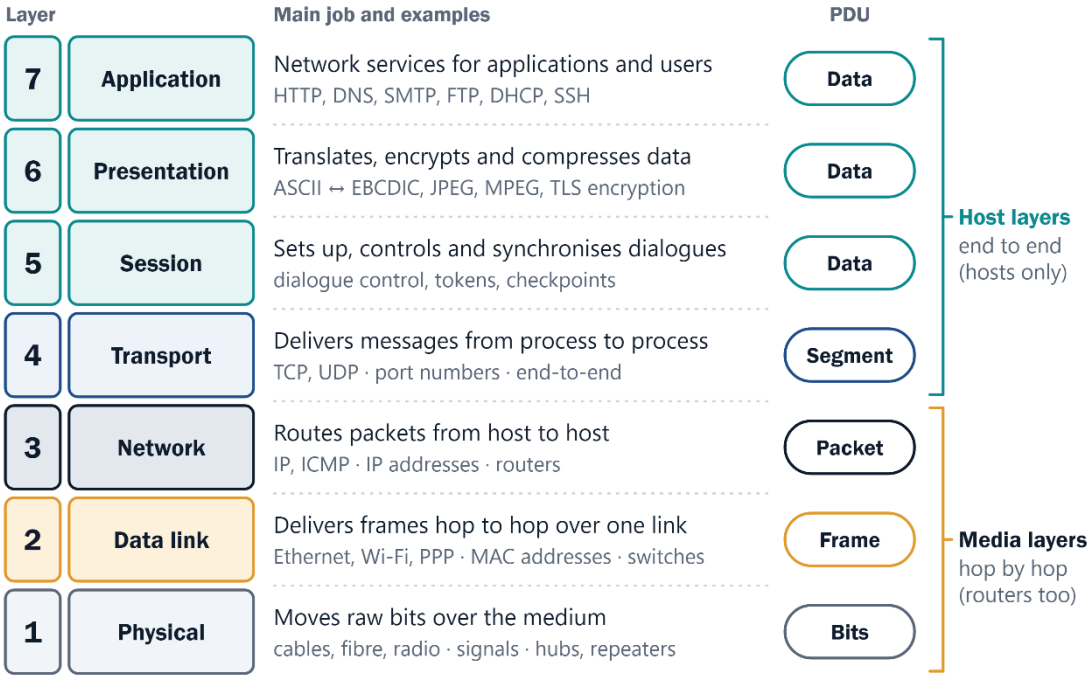


Figure 1.4 The seven OSI layers, with the main job, examples and the PDU of each layer.

OSI is a **reference model**, not a network architecture: it says *what* each layer should do, but does not fix the exact protocols. The OSI protocols did not succeed commercially, but the model became the common language

of networking ("a layer-2 switch", "a layer-7 firewall"). Layers are numbered from the bottom. A memory aid from layer 7 down: "**A**ll **P**eople **S**eem **T**o **N**eed **D**ata **P**rocessing".

We describe the layers from the bottom up, the order of this course. A sender's data travel down the stack; a receiver's data travel up.

Layer 1, physical. The physical layer moves individual **bits** over a medium: copper, fibre or radio. It defines connectors, voltage or light levels, how a bit becomes a signal, the data rate, bit synchronisation and the transmission mode (simplex, half- or full-duplex). Devices: cables, repeaters, hubs. Examples: 1000BASE-T Ethernet over copper, the Wi-Fi radio, the Bluetooth radio (Lecture 2).

Layer 2, data link. The data link layer moves **frames** from one node to the next over a single link: **node-to-node** (hop-to-hop) delivery. It builds frames, puts the 48-bit **MAC addresses** in the header (on a LAN), and adds a **frame check sequence (FCS)** in the trailer so that damaged frames are detected and discarded. It also does flow control and decides who may use a shared medium. Devices: network cards, switches, access points. Protocols: Ethernet, Wi-Fi, PPP, HDLC (Lectures 3–5).

Layer 3, network. The network layer delivers **packets** from the source host to the destination host across many networks: **host-to-host** delivery. It gives every host a logical **IP address** (32 bits in IPv4, 128 bits in IPv6) and uses **routing** to choose the path. Each router forwards the packet one hop closer to the destination. Devices: routers, layer-3 switches. Protocols: IPv4, IPv6, ICMP; routing protocols such as RIP and OSPF serve this layer (Lectures 5, 6, 8, 9).

Layer 4, transport. The transport layer delivers a whole message from one **process** (running program) to another, e.g. from your browser to the web-server program: **process-to-process** or end-to-end delivery. It identifies processes by 16-bit **port numbers**. It cuts long messages into **segments** and puts them back in order at the receiver, and it gives end-to-end error and flow control. Protocols: TCP, which does all of these jobs, and UDP, which adds little more than the port numbers (Lectures 10, 11).

Layer 5, session. The session layer sets up, manages and ends the **dialogue** between two applications. It controls whose turn it is to send and adds **checkpoints** to long transfers. Example: 1,200 pages are sent with a checkpoint every 50 pages, and the link fails during page 437. The last checkpoint was after page 400, so only pages 401–437 (37 pages) are sent again.

Layer 6, presentation. The presentation layer lets the receiver understand the data even when the two machines represent data differently. Its jobs are **translation** (e.g. between the ASCII and EBCDIC character codes), **encryption**, and **compression** (e.g. JPEG for images). Details: Lecture 12.

Layer 7, application. The application layer gives users and programs access to network services: web, e-mail, file transfer, name lookup. Protocols: HTTP, SMTP, FTP, DNS, DHCP, SSH, Telnet (Lectures 12–15). The browser program itself is not part of the layer; the HTTP protocol it uses is. The PDU of layers 5–7 is simply called **data**.

> **NOTE**
>
> Layers 1–3 are the **media layers**: they work **hop by hop**, and every router runs them. Layers 4–7 are the **host layers**: they work **end to end** and run only in the two end hosts. So a hub works at layer 1, a switch at layers 1–2 (it reads MAC addresses), a router at layers 1–3 (it reads IP addresses) and a host at all seven.

THINK ABOUT IT

In Lab 1 (Packet Tracer, Simulation mode), click a PDU envelope at a switch and at a router and open the **OSI Model** tab. Which layers does each device process? Why does the router stop at layer 3?

1.5.1 OSI and the TCP/IP model

The Internet runs the **TCP/IP protocol suite**, not the OSI protocols. Here the protocols came first and the model was written later. RFC 1122 defines **four layers**: application (OSI 5–7), transport (OSI 4), internet (OSI 3) and link (OSI 1–2). Textbooks such as Kurose and Forouzan use a **five-layer** version that keeps physical and link apart; both views are correct. Session and presentation jobs are done inside the applications or by TLS (e.g. HTTPS encryption). Full comparison: Lecture 15.

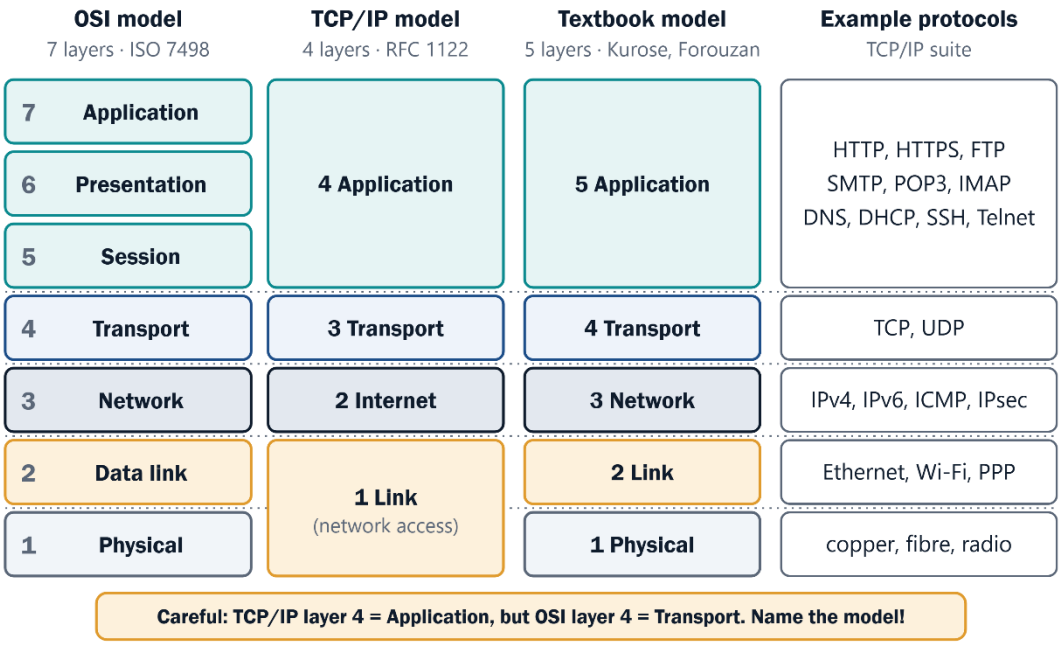


Figure 1.5 The OSI model, the four-layer TCP/IP model of RFC 1122 and the five-layer textbook model, with example protocols.

1.6 PDUs, encapsulation and decapsulation

Recall that a PDU is the block of data that a layer exchanges with its peer. Every PDU has two parts. The **header** holds control information for the peer (addresses, a type field, a sequence number, a length). The **payload** is the whole PDU from the layer above. The data link layer also adds a **trailer**, the FCS, at the end, so the hardware can compute it while the bits are sent.

COMMON MISTAKE

Several lecture notes expand PDU as "*Packet* Data Unit". This is wrong. PDU means **Protocol Data Unit**; "packet" is only the name of the layer-3 PDU.

When a browser sends a request, each layer wraps the PDU from above, like a letter put into an envelope. This is **encapsulation**:

1. The application creates the message, e.g. `GET /index.html HTTP/1.1`.
2. Transport adds a TCP header (source port 49152, destination port 80, sequence number…): a **segment**.
3. Network adds an IP header (source and destination IP addresses, time to live (TTL), Protocol = 6 for TCP): a **packet**.
4. Data link adds an Ethernet header (destination and source MAC addresses, EtherType `0x0800` for IPv4) and a 4-byte FCS: a **frame**.
5. Physical sends the frame as **bits**, after the preamble and SFD.

No layer looks inside its payload. The figure shows the sizes when the application hands 1,460 bytes to TCP.

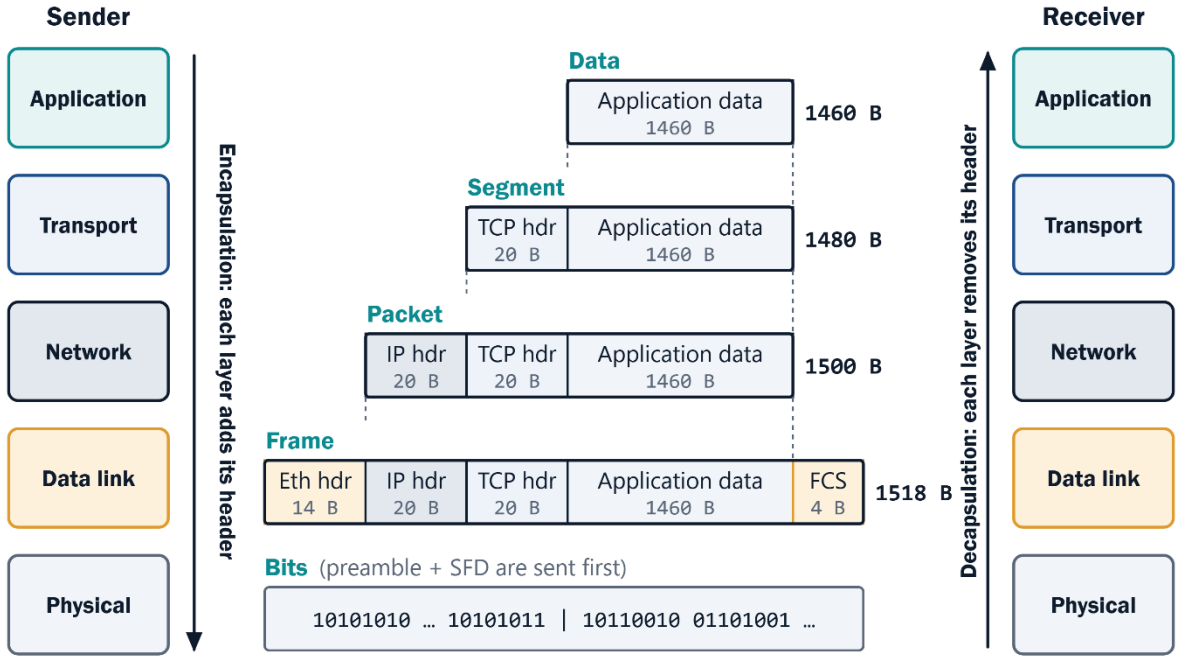


Figure 1.6 Encapsulation of 1,460 bytes of web data over TCP, IPv4 and Ethernet. Each layer adds a header; the data link layer also adds the FCS trailer.

Table 1.4 PDU names and header sizes in a TCP/IP stack over Ethernet.

Layer	PDU name	What the layer adds	Size
Application (OSI 5–7)	data (also: message)	application header	varies
Transport	segment (TCP); user datagram (UDP)	TCP header / UDP header	20–60 B / 8 B
Network	packet (also: datagram)	IPv4 header (IPv6 header)	20–60 B (40 B)
Data link	frame	Ethernet header + FCS trailer	14 B + 4 B
Physical	bits	preamble + start-of-frame delimiter (SFD)	8 B

At the receiver the steps run in reverse: this is **decapsulation**. Each layer checks and removes its own header and passes the payload up; a frame that fails the FCS check is discarded. To choose the protocol above, each layer reads a type field in its header. This is **demultiplexing**:

- layer 2 reads the **EtherType**: `0x0800` = IPv4, `0x86DD` = IPv6, `0x0806` = ARP;

- layer 3 reads the IP **Protocol** field: 1 = ICMP, 6 = TCP, 17 = UDP;
- layer 4 reads the **destination port**: 80 = web server, 53 = DNS server.

Headers (and the trailer) carry no user data, so they reduce the efficiency. The share of the bytes that are not payload is the **overhead**:

$$\text{efficiency } \eta = \text{payload bytes} \div \text{total bytes sent}$$

$$\text{overhead} = (\text{total bytes} - \text{payload bytes}) \div \text{total bytes sent} = 1 - \eta$$

WORKED EXAMPLE 1.1: ENCAPSULATION SIZES AND EFFICIENCY

Problem. A web server sends 1,460 B of HTTP data in one TCP segment over IPv4 and Ethernet at 100 Mbps. Find the segment, packet and frame sizes, the time to send the frame and the efficiency. Repeat for 10 B of data sent over UDP.

1. Segment = 1,460 + 20 (TCP header) = **1,480 B**.
2. Packet = 1,480 + 20 (IPv4 header) = **1,500 B**, the largest payload an Ethernet frame can carry.
3. Frame = 1,500 + 14 (header) + 4 (FCS) = **1,518 B** = 12,144 bits.
4. Time = 12,144 bits ÷ 100,000,000 bps = **121.44 µs**.
5. η = 1,460 ÷ 1,518 = **96.2 %**.

Small message: 10 + 8 (UDP) + 20 (IPv4) = 38 B. Ethernet needs at least 46 B of payload, so it adds 8 B of padding: frame = 46 + 18 = **64 B** (the minimum frame) and η = 10 ÷ 64 = **15.6 %**. Headers cost the same for every message, so small messages waste most of the capacity.

A frame carries three kinds of address. The **MAC address** (layer 2) serves node-to-node delivery and **changes at every router**. The **IP address** (layer 3) serves host-to-host delivery and stays the same end to end, unless address translation (NAT) is used. The **port** (layer 4) serves process-to-process delivery and also stays the same. A port is only a number, whereas a **socket** is an end point identified by an IP address *and* a port.

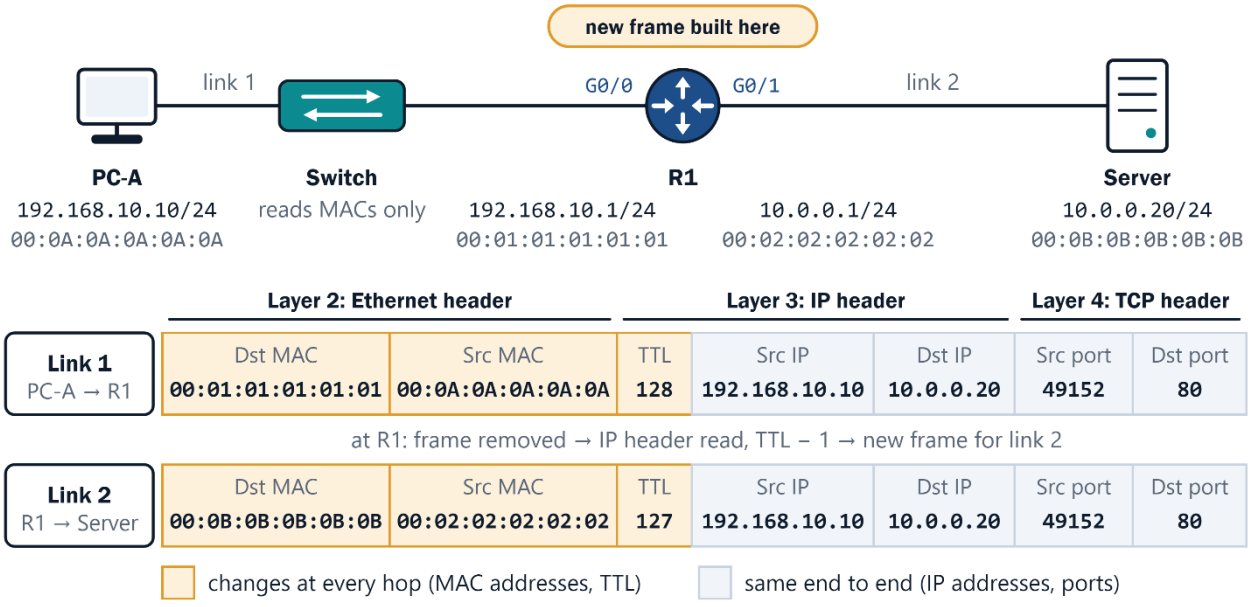


Figure 1.7 Worked example 1.2: the MAC addresses and the TTL change at router R1; the IP addresses and ports do not.

WORKED EXAMPLE 1.2: WHICH ADDRESSES CHANGE ON THE WAY?

Problem. PC-A (192.168.10.10) opens a web page on a server (10.0.0.20, port 80) from port 49152, through a switch and router R1 (figure above). PC-A sets the TTL to 128. Which addresses does the frame carry on each link?

1. The server is on another network, so PC-A sends the frame to its default gateway: the destination MAC is R1's G0/0 address. PC-A finds it with ARP, the Address Resolution Protocol (Lecture 5).
2. The switch reads only the Ethernet header and forwards the frame unchanged.
3. R1 removes the frame, reads the IP header, lowers the TTL from 128 to 127 and chooses interface G0/1.
4. R1 builds a **new frame** from its G0/1 MAC to the server's MAC: both MAC addresses change.
5. The IP addresses (192.168.10.10 → 10.0.0.20) and ports (49152 → 80) are **unchanged**.

SUMMARY

- A **protocol** fixes the format and order of messages and the actions taken on them.
- Layer *n* offers a **service** to layer *n*+1 (vertical) and talks to its **peer** through a **protocol** (horizontal).
- Primitives: **request, indication, response, confirm** (confirmed service: all four; unconfirmed: the first two).
- **Connection-oriented** = phone call; **connectionless** = post. Reliability is a separate property.
- OSI PDUs: bits, frame, packet, segment, then data in layers 5–7. TCP/IP has four layers (RFC 1122).
- **Encapsulation** adds headers going down; **decapsulation** removes them going up. MAC addresses change at every router; IP addresses and ports do not.

1.7 Review questions

1. Define *protocol* and name its three elements.
2. Give two benefits and two costs of layering. What is a *peer*?
3. Distinguish *service*, *interface* and *protocol*. Which is vertical and which horizontal?
4. Name the four OSI primitives in the order used by a confirmed service and match each to a step of a telephone call. Which primitives does an unconfirmed service use?
5. Compare connection-oriented and connectionless service in four points, with one protocol of each type. Why is an unreliable service sometimes preferred?
6. Which OSI layer: (a) turns bits into signals; (b) chooses the route; (c) builds frames; (d) inserts checkpoints; (e) converts ASCII to EBCDIC; (f) delivers a message to the right process; (g) adds the MAC addresses; (h) gives users e-mail? Then name the PDU of each layer, and say which layer adds a trailer and why.
7. Explain encapsulation and decapsulation. How does the receiver know which upper protocol gets the payload? Give the field and one value at layers 2, 3 and 4.
8. Messages of 500 B pass down through 4 layers, and each layer adds a 25 B header. What fraction of the transmitted bytes is header?

9. A host sends 200 B of data in one UDP datagram over IPv4 and Ethernet at 100 Mbps. Find the sizes of the user datagram, packet and frame, the frame's transmission time and the efficiency.
10. A web request travels PC-A → switch 1 → R1 → R2 → switch 2 → server. (a) How many times is a new frame built, and by which devices? (b) Do the IP addresses change? (c) By how much does the TTL fall? (d) Do the port numbers change?

1.8 References

- J. F. Kurose and K. W. Ross, *Computer Networking: A Top-Down Approach*, 7th ed., Pearson, 2017: §1.1.3 and §1.5.
- A. S. Tanenbaum and D. J. Wetherall, *Computer Networks*, 5th ed., Pearson, 2011: §1.3 and §1.4.
- B. A. Forouzan, *Data Communications and Networking*, 5th ed., McGraw-Hill, 2013: Ch. 2.
- ISO/IEC 7498-1:1994 = ITU-T X.200 (OSI model); ITU-T X.210 (service conventions); RFC 1122; RFC 791 (IPv4); RFC 768 (UDP); RFC 9293 (TCP).
- Lecture material consulted: M. I. Shujaa (Middle Technical University); M. S. Mahmoud (Al-Mansour University College); L. A. Hassnawi (Al-Mustaqbal University); N. H. Hussein (Iraqi University of Sciences); Al-Mustafa University; Imam Al-Kadhim College; Al-Rafidain University.